

kubernetes svc cluster ip iptables规则跟踪

产品版本：

ESCloud 5.0.3

问题描述：

svc iptables规则链

Jira链接：

无

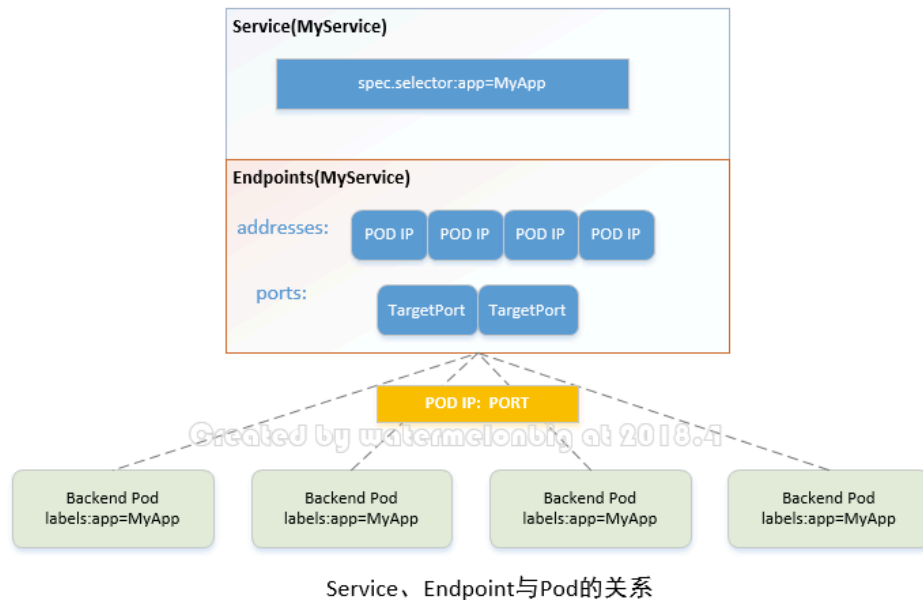
解决方案：

service概念：

- Kubernetes Service定义了这样一种抽象：一个Pod的逻辑分组，一种可以访问它们的策略 —— 通常称为微服务。这一组Pod能够被Service访问到，通常是通过Label Selector
 - Service能够提供负载均衡的能力，但是在使用上有以下限制：
 - 只提供 4 层负载均衡能力，而没有 7 层功能，但有时我们可能需要更多的匹配规则来转发请求，这点上 4 层负载均衡是不支持的
1. service通过label选择对应pod
 2. 每个service分配一个cluster ip，集群内部访问是通过servicename：
 - a. 通过coredns组件对svc解析
 - b. 使用ingress，通过ServiceName和ServicePort访问
 3. Service 在 K8s 中有以下四种类型
 - a. ClusterIP 【内部访问】
 - b. NodePort 【外部访问】
 - c. LoadBalancer 【外部负载均衡】
 - d. ExternalName 【直达模式】

下面主要介绍service通过cluster ip访问，对应kubernetes节点上的iptables规则

clusterIP主要在每个node节点使用iptables【新版本模式是ipvs代理模式因此此处为ipvs，代理模式不同所使用的底层方案也是不一致的】，将发向clusterIP对应端口的数据，转发到kube-proxy中。然后kube-proxy自己内部实现有负载均衡的方法，并可以查询到这个service下对应pod的地址和端口，进而把数据转发给对应的pod的地址和端口



Endpoints表示了一个Service对应的所有Pod副本的访问地址，而Endpoints Controller负责生成和维护所有Endpoints对象的控制器。它负责监听Service和对应的Pod副本的变化。

- 如果监测到Service被删除，则删除和该Service同名的Endpoints对象；
- 如果监测到新的Service被创建或修改，则根据该Service信息获得相关的Pod列表，然后创建或更新Service对应的Endpoints对象。
- 如果监测到Pod的事件，则更新它对应的Service的Endpoints对象。

kube-proxy进程获取每个Service的Endpoints，实现Service的[负载均衡](#)功能。

Service Controller

Service Controller是属于kubernetes集群与外部的云平台之间的一个接口控制器。Service Controller监听Service变化，如果是一个LoadBalancer类型的Service，则确保外部的云平台上对该Service对应的LoadBalancer实例被相应地创建、删除及更新路由转发表。

主要需要以下几个组件的协同工作：

- **apiserver** 用户通过kubectl命令向apiserver发送创建service的命令，apiserver接收到请求后将数据存储到etcd中

- **kube-proxy kubernetes** 的每个节点中都有一个叫做kube-proxy的进程，这个进程负责感知service，pod的变化，并将变化的信息写入本地的iptables规则中
- **iptables** 使用NAT等技术将virtualIP的流量转至endpoint中

看下具体规则：

```
1 [root@node-5 ~]# kubectl -n openstack get svc | grep nova
2 nova ExternalName <none> ingress.kube-system.svc.cluster
3 nova-api ClusterIP 10.222.87.232 <none>
4 nova-metadata ClusterIP 10.222.180.173 <none>
5 nova-novncproxy ClusterIP 10.222.208.5 <none>
6
```

```
1 [root@node-5 ~]# iptables-save | grep 10.222.87.232
2 -A KUBE-SERVICES ! -s 10.232.0.0/15 -d 10.222.87.232/32 -p tcp -m comment --comment "openstack
3 -A KUBE-SERVICES -d 10.222.87.232/32 -p tcp -m comment --comment "openstack/nova-api:n-api clu
4
5 # 如果是cluster内的pod (源地址来自10.232.0.0/15) 要访问nova-api, 则允许
6 # 其他地址访问nova-api, 则跳转到KUBE-SVC-NY6F7ZLPRV77PAQA
```

看下 **KUBE-SVC-NY6F7ZLPRV77PAQA** 规则

```
1 [root@node-5 ~]# iptables-save | grep KUBE-SVC-NY6F7ZLPRV77PAQA
2 :KUBE-SVC-NY6F7ZLPRV77PAQA - [0:0]
3 -A KUBE-SERVICES -d 10.222.87.232/32 -p tcp -m comment --comment "openstack/nova-api:n-api cl
4 -A KUBE-SVC-NY6F7ZLPRV77PAQA -m statistic --mode random --probability 0.33332999982 -j KUBE-S
5 -A KUBE-SVC-NY6F7ZLPRV77PAQA -m statistic --mode random --probability 0.50000000000 -j KUBE-S
6 -A KUBE-SVC-NY6F7ZLPRV77PAQA -j KUBE-SEP-PAHCJ5S6JJUDTMF3
7
8 # 1/2的概率跳转到KUBE-SEP-JAPCWP3QRY4BUGDC
9 # 1/3的概率跳转到KUBE-SEP-UF4ND3T07RIQWJM4
10 # 剩下的概率跳转到KUBE-SEP-PAHCJ5S6JJUDTMF3
```

接着看下最大概率的 **KUBE-SEP-JAPCWP3QRY4BUGDC** 规则

```
1 [root@node-5 ~]# iptables-save | grep KUBE-SEP-JAPCWP3QRY4BUGDC
2 :KUBE-SEP-JAPCWP3QRY4BUGDC - [0:0]
3 -A KUBE-SEP-JAPCWP3QRY4BUGDC -s 10.10.3.7/32 -j KUBE-MARK-MASQ
4 -A KUBE-SEP-JAPCWP3QRY4BUGDC -p tcp -m tcp -j DNAT --to-destination 10.10.3.7:8774
5 -A KUBE-SVC-NY6F7ZLPRV77PAQA -m statistic --mode random --probability 0.50000000000 -j KUBE-S
6
7 # 源地址是 10.10.3.7/32 则允许
8 # 其他地址 dnat到 10.10.3.7/32
```

看看 **10.10.3.7** 这个地址是什么地址

```
1 [root@node-5 ~]# ssh 10.10.3.7
2 Warning: Permanently added '10.10.3.7' (ECDSA) to the list of known hosts.
3 Last login: Fri Apr 22 06:04:21 2022 from 10.255.252.2
4 Authorized users only. All activity may be monitored and reported
5 [root@node-4 ~]# ip a | grep 10.10.3.7
6     inet 10.10.3.7/24 brd 10.10.3.255 scope global br-roller
7
8
```

访问nova-api，最终访问node-4节点 10.10.3.7